COEP Technological University

A Unitary Public University if Government of Maharashtra w.e.f. 21st June 2022

(Formerly College of Engineering Pune)

Department of Electronics and Telecommunication

Mini Project Report **(2022-23)**

Title : Data Acquisition system for Vehicle

Done by :

1. 112007021-Gaurav Kolte
2. 112007022-Neha B Gawali
3. 112007011- Bhagyashree Bhandarkar
4. 112007042- Mohit A. Panpaliya

Guided By :

Dr. S. P. Mahajan

Prof. Swapnil Mali Sir

INDEX

Sr. No	Topics	Page No.
1.	Abstract	3
2.	Introduction	3
3.	Hardware	5
4.	Block Diagram	9
5.	Workflow	10
6.	Code	10
7.	Implementation	22
8.	Challenges Faced	22
9.	Conclusion	22

Abstract

This project aims to utilize the capabilities of Arduino Mega and ESP8266 to collect data from sensors installed in a car and transmit that data to the Google Cloud Platform for analysis. The collected data is then analyzed using machine learning algorithms to provide real-time insights into the car's performance, allowing for informed decisions on maintenance and optimization. The project involves selecting appropriate sensors, collecting and processing data, transmitting data to the cloud, setting up the cloud platform for storage and analysis , and visualizing the insights gained. The insights gained from the analysis can be used to optimize the car's performance and improve its efficiency, leading to reduced costs and improved safety.

Introduction

The aim of this project is to design and develop a data acquisition system for a car using Arduino Mega, ESP8266, Arduino Uno, GPS, load cell, speed sensor, and accelerometer. The system will collect data from these sensors and analyze them to provide valuable insights into the car's performance.

This project report will describe the design and implementation of the system, as well as the data analysis techniques used to extract meaningful information from the collected data.

Need :

- To validate the design of the current vehicle that is the suspension geometry .
- To verify the Motor is being used to its full capacity which it is designed.

Hardware

- **NodeMCU – ESP8266**
  With the complete and self-contained Wi-Fi networking capabilities, ESP8266 can perform either as a standalone application or as the slave to a host MCU. When ESP8266 hosts the application, it promptly boots up from the flash. The integrated highspeed cache helps to increase the system performance and optimize the system memory. Also, ESP8266 can be applied to any microcontroller design as a Wi-Fi adaptor through SPI/SDIO or UART interfaces. ESP8266 integrates antenna switches, RF balun, power amplifier, low noise receiveamplifier, filters and power management modules. The compact design minimizes the PCB size and requires minimal external circuitries. It can be interfaced with external sensors and other devices through the GPIOs.

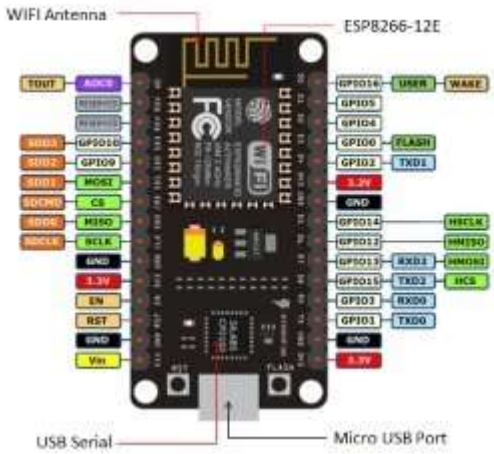


Fig. NodeMCU – ESP8266

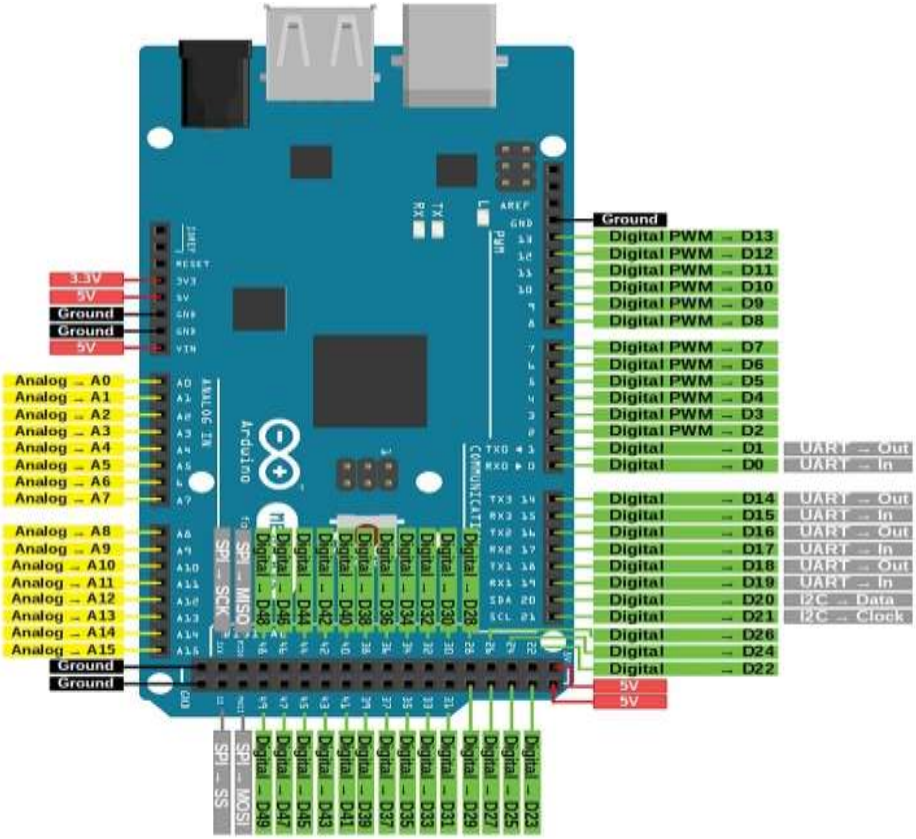
Item	Operating temperature Range	Wi-Fi Frequency range	Working Current	Operating Voltage	Protocols
ESP8266	-40 to 125 degreeC	2.4 to 2.5 GHz	80 mA avg.	2.5 – 3.6 V	802.11 b/g/n (HT20)

Arduino Mega 2560 : The Arduino Mega 2560 is a microcontroller board based on the ATmega2560. It has 54 digital input/output pins (of which 15 can be used as PWM outputs), 16 analog inputs, 4 UARTs (hardware serial ports), a 16 MHz crystal oscillator, a USB connection, a power jack, an ICSP header, and a reset button.

EEPROM : The ATmega2560 features 4kb (4096 bytes) of EEPROM, a memory which is not erased when powered off.

54 digital & 16 analog pins : The Mega 2560 has 54 digital pins, whereas 15 supports PWM, and 16 analog input pins.

Four serial ports : Connect to several devices through the 4x hardware serial ports (UARTs) to your Arduino Mega.



Arduino GY-NEO6MV2 GPS Module c/w Antenna & Flight Control EEPROM

GY-NEO6MV2 board features the u-blox NEO-6M GPS module with antenna and built-in EEPROM. This is compatible with various flight controller boards designed to work with a GPS module

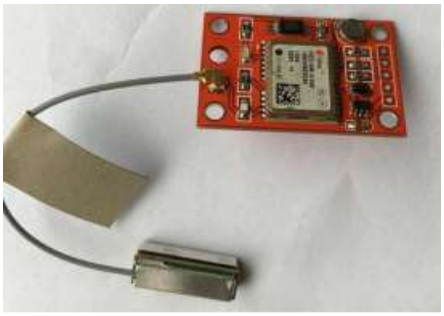


Fig. NEO6MV2 GPS Module

- **Technical Specifications:**
  - Power Supply Range: 3 V to 5 V
  - Model: GY-GPS6MV2
  - Ceramic antenna
  - EEPROM for saving the configuration data when powered off
  - Backup battery

- LED signal indicator
- Mounting Hole Diameter: 3 mm
- Default Baud Rate: 9600 bps
- Module size 23mm * 30mm
- Antenna size 12 * 12mm
- Cable:20mm

- **Features**:
  - Use XM37-1612 module, MTK Platform, with high-gain active antenna
  - TTL level, compatible with 3.3V/5V system
  - The default baud rate: 9600
  - With rechargeable backup battery, can save the ephemeris data when it power down, and make the warm start.
  - Suitable for RC quad copter, navigator

ADXL 345 Accelerometer Module

ADXL345 Module Features & Specifications

3V-6V DC Supply Voltage

- On-board LDO Voltage regulator
- Built-in Voltage level convertor (MOSFET based)
- It can be interface with 3.3V or 5V Microcontroller.
- Ultra-Low Power: 40uA in measurement mode, 0.1uA in standby@ 2.5V
- Tap/Double Tap Detection
- Free-Fall Detection
- SPI and I2C interfaces

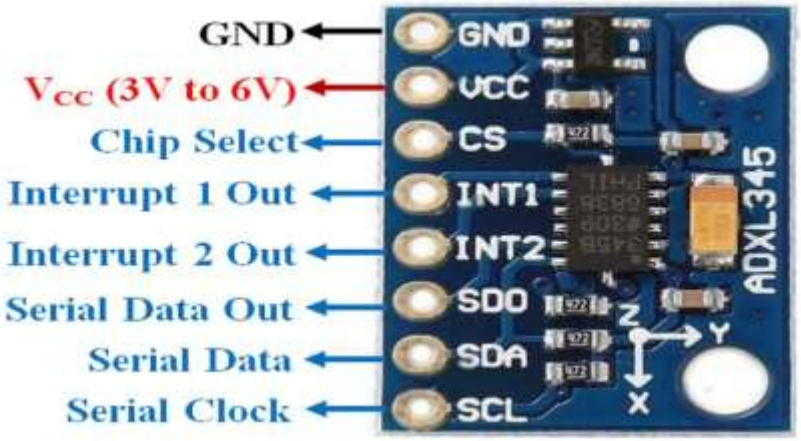


Fig. ADXL 345

- Measuring Range: ±16g
  - Measuring Values (-16g to +16g):
- X: -235 to +270
- Y: -240 to +260
- Z: -240 to +270

S type Load Cell capacity(1 ton) :

Standard Specification :	
Nominal Output	3.0mV/V (mili Volt per Volt) & 2.0 mV/V
Excitation Voltage	9 -12 VDC (Maximum 15 VDC)
Safe Load	150% of Rated Capacity
Ultimate Load	200% of Rated Capacity
Allowable Side Load	50% of Rated Capacity
Non Linearity	<±0.025% of FSO (Full Scale Output)
Creep (30 minutes)	<±0.030% of FSO
Non-Repeatability	≤±0.020% of FSO
Zero Balance	<±1% of FSO
Input Resistance	350 ± 5 ohms
Output Resistance	350 ± 5 ohms
Insulation resistance	Above 5000 Mega ohms
Temp. Compensation	-10°C to +40°C
Temp. Effect on Zero	≤± 0.030% of FSO /10 °C
Material	Electroless Nickle Plated Tool Steel
Environmental Protection Class	IP67.
Connection	Input : +(Red) -(Black) Output : +(Green) -(White)

Fig. S Type Load cell Capacity(1 Ton)

Software :

- MATLAB
- ARDUINO IDE
- GOOGLE CLOUD

Matlab :

MATLAB is a powerful tool for processing and analyzing such data. In this section, we will discuss the steps involved in processing acceleration and load cell data in MATLAB.

Reading data from Excel:

The first step is to read the data from an Excel file. MATLAB provides several functions to read data from Excel files, such as readtable and xlsread. These functions can be used to read the data from the Excel file into a MATLAB table or matrix.

Extracting acceleration and load cell data:

Once the data is read into MATLAB, the next step is to extract the acceleration and load cell data. In the case of acceleration data, the x, y, and z components of the acceleration can be extracted from the table or matrix. In the case of load cell data, the load cell readings can be extracted.

Processing acceleration data:

The acceleration data can be processed in several ways depending on the application. For example, the acceleration data can be converted to velocity or displacement data using

integration. The acceleration data can also be used to calculate torque or force using appropriate equations.

Processing load cell data:

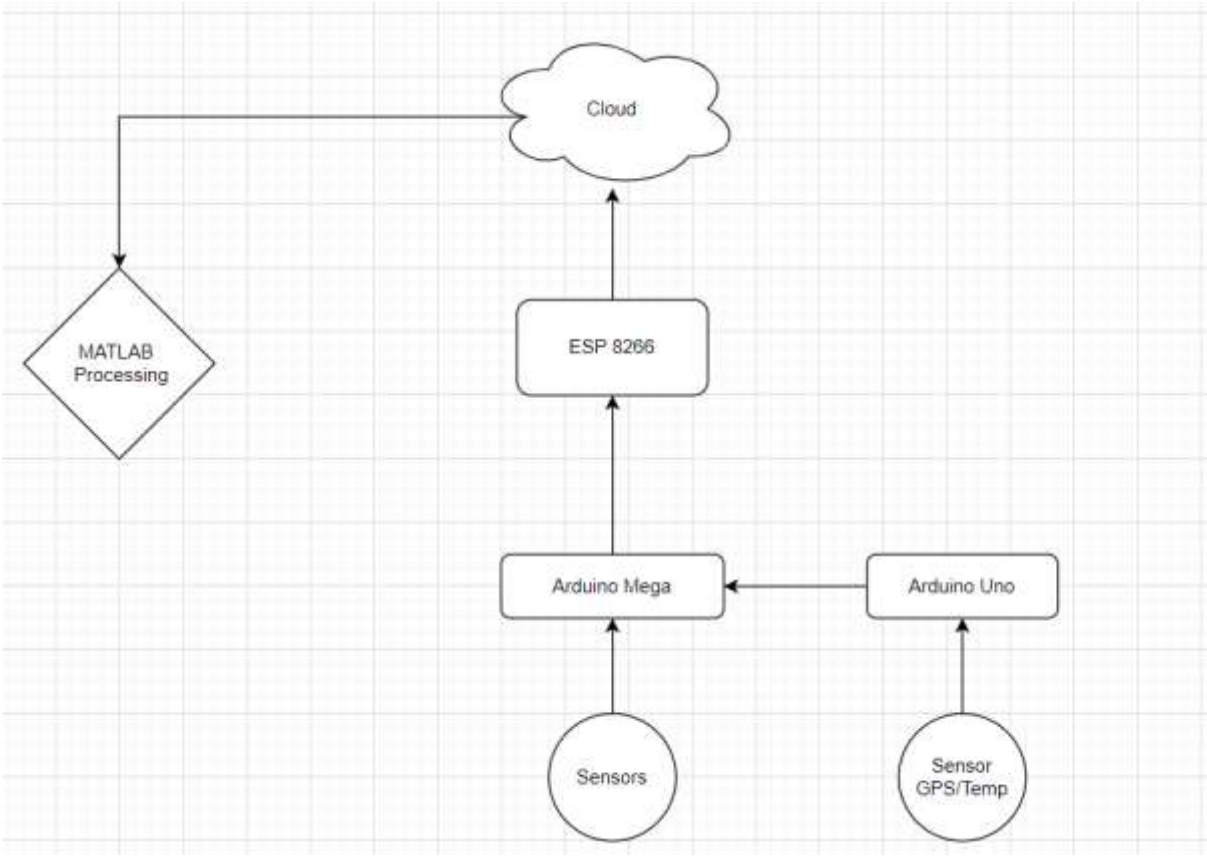
The load cell data can be processed by applying calibration factors to convert the load cell readings to actual load values. The load cell data can also be analyzed to calculate statistical parameters such as mean, standard deviation, and variance.

Plotting data:

Once the data is processed, it can be plotted to visualize the trends and patterns in the data. MATLAB provides several functions to plot data, such as plot, scatter, and histogram. These functions can be used to plot the acceleration and load cell data.

Arduino Software (IDE) - The Arduino Integrated Development Environment - or Arduino Software (IDE) - contains a text editor for writing code, a message area, a text console, a toolbar with buttons for common functions and a series of menus. It connects to the Arduino hardware to upload programs and communicate with them.

Block Diagram



Workflow

The following are the steps in the workflow for the project "Data Acquisition and Analysis for Car Performance Using Arduino Mega and ESP8266 and Google Cloud":

Sensor Selection and Installation : The first step we did is to choose the appropriate sensors to monitor the performance of the vehicle. Temperature sensors, Load cell sensors, and accelerometers are sensors that we used. After that, the sensors are installed in the vehicle.

Data Collection from Sensors : The sensors Load cell, accelerometer are linked to the Arduino Mega,and thermistor and GPS is connected to Arduino uno which is then eventually connected to Arduino Mega which collects data from them. Temperature, Location, and acceleration data can all be collected.

Data Processing and Transmission : The Arduino Mega collects data, which is then processed and converted into a format suitable for transmission to the Google Cloud Platform. The data is transmitted using the ESP8266 module.

Google Cloud Platform Setup : The Google Cloud Platform is set up for data storage and analysis. This involves creating a project, setting up the Cloud IoT Core, creating a Pub/Sub topic, and setting up a dataflow pipeline to store the data in BigQuery.

Data Analysis : The collected data is analyzed in Matlabto provide insights into the car's performance of Torque, RPM . Examples of analysis include Validation of Suspension Geometry.

Visualization and Reporting : The insights gained from the data analysis are visualized and reported in a way that is easy to understand. Examples of visualization include charts, graphs, and dashboards.

Maintenance and Optimization : The insights gained from the data analysis can be used to optimize the car's performance and improve its efficiency. Examples of optimization include adjusting the car's engine, changing the tire pressure, and changing the car's driving habits.

In summary, the workflow for the project involves selecting and installing sensors, collecting and processing data, transmitting the data to the Google Cloud Platform, setting up the platform for data storage and analysis, analyzing the data, visualizing and reporting the insights, and using the insights to optimize the car's performance.

<u>Code</u>

ESP8266 code:

```
#include <ESP8266WiFi.h>

#include <WiFiClientSecure.h>

#include "SoftwareSerial.h"


const int RX_pin = 13; // for ESP8266 use 13  D7 on NodeMCU/wemos-d1-esp8266

const int TX_pin = 12; // for ESP8266 use 12  D6  on NodeMCU/wemos-d1-esp8266

// else you need to define your own RX_pin, TX_pin

SoftwareSerial softSerial(RX_pin, TX_pin);


const char* ssid = "Hello"; //--> Your wifi name or SSID.

const char* password = "enter123"; //--> Your wifi password.
```

```
//----------------------------------------Host & httpsPort
const char* host = "script.google.com";
const int httpsPort = 443;
//----------------------------------------
String line;

WiFiClientSecure client; //--> Create a WiFiClientSecure object.

String GAS_ID = "AKfycbxsKEzAZjHwol_Fj-Dy1hOHqpOIOk72Tjc4MdHm4BrBIvBkeZPXNFZq35CXctmekA8"; //--> spreadsheet script ID

void setup() {
  // put your setup code here, to run once:
  Serial.begin(115200);
  softSerial.begin(115200);
  delay(500);

  WiFi.begin(ssid, password); //--> Connect to your WiFi router
  Serial.println("");

  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(250);
  }
  Serial.println("");
  Serial.print("Successfully connected to : ");
  Serial.println(ssid);
  Serial.print("IP address: ");
```

```
  Serial.println(WiFi.localIP());
  Serial.println();

}

void loop() {
  if(softSerial.available() > 0){
    String request = softSerial.readStringUntil('\n');
    sendData(request);
  }
}

void sendData(String data) {
  client.setInsecure();
  Serial.println("==========");
  Serial.print("connecting to ");
  Serial.println(host);
  if (!client.connect(host, httpsPort)) {
    Serial.println("connection failed");
    return;
  }

  String url = "/macros/s/" + GAS_ID + data;
  Serial.print("requesting URL: ");
  Serial.println(url);

  client.print(String("GET ") + url + " HTTP/1.1\r\n" +
        "Host: " + host + "\r\n" +
        "User-Agent: BuildFailureDetectorESP8266\r\n" +
```

```
        "Connection: close\r\n\r\n");

  Serial.println("request sent");
  while (client.connected()) {
    line = client.readStringUntil('\n');
    if (line == "\r") {
      Serial.println("headers received");
      break;
    }
  }
  line = client.readStringUntil('\n');
  if (line.startsWith("{\"state\":\"success\"")) {
    Serial.println("esp8266/Arduino CI successfull!");
  } else {
    Serial.println("esp8266/Arduino CI has failed");
  }
  Serial.print("reply was : ");
  Serial.println(line);
  Serial.println("closing connection");
  Serial.println("==========");
  Serial.println();
}
```

Ardiuno code :

```
#include <SoftwareSerial.h>
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_ADXL345_U.h>
#include <Q2HX711.h>
#include <SPI.h>
```

```
#include <SD.h>

Adafruit_ADXL345_Unified accel = Adafruit_ADXL345_Unified();


SoftwareSerial espSerial(14, 15); //TX-14    RX - 15

const byte hx711_data_pin = 38;
const byte hx711_clock_pin = 39;
const byte hx711_data_pin_2 = 40;
const byte hx711_clock_pin_2 = 41;
const byte hx711_data_pin_3 = 42;
const byte hx711_clock_pin_3 = 43;
const byte hx711_data_pin_4 = 44;
const byte hx711_clock_pin_4 = 45;

float error_load_cell;
float error_load_cell_2;
float error_load_cell_3;
float error_load_cell_4;

float load1;
float load2;
float load3;
float load4;

Q2HX711 hx711(hx711_data_pin, hx711_clock_pin);
Q2HX711 hx711_2(hx711_data_pin_2, hx711_clock_pin_2);
Q2HX711 hx711_3(hx711_data_pin_3, hx711_clock_pin_3);
```

```
Q2HX711 hx711_4(hx711_data_pin_4, hx711_clock_pin_4);



int chipSelect = 53;


void setup()
{
  Serial.begin(9600);
  Serial1.begin(9600);
  espSerial.begin(115200);
  for(int i=0;i<5;i++){
    error_load_cell=(hx711.read()/500.0);
    error_load_cell_2=(hx711_2.read()/500.0);
    error_load_cell_3=(hx711_3.read()/500.0);
    error_load_cell_4=(hx711_4.read()/500.0);
  }
  Serial.println(error_load_cell);
  Serial.println(error_load_cell_2);
  Serial.println(error_load_cell_3);

  Serial.println(error_load_cell_4);
  accel.begin();
  Serial.print("Initializing SD Card...");
  if (!SD.begin(chipSelect))
  {
    Serial.println("Card failed, or not present");
    //while(1);
  }
  Serial.println("Card Initialised");
```

```
}

void loop()
{

  /*if (Serial1.available() > 0){
    Serial.print("Serial1 available");
    if (gps.encode(Serial1.read())){
      displayInfo();
    }
  }*/
  // GPS Code End

    sensors_event_t event;
    accel.getEvent(&event);
    Serial.print("X: "); Serial.print(event.acceleration.x); Serial.print("  ");
    Serial.print("Y: "); Serial.print(event.acceleration.y); Serial.print("  ");
    Serial.print("Z: "); Serial.print(event.acceleration.z); Serial.print("  ");
    Serial.println("m/s^2 ");

    File dataFile = SD.open("accelerometer.txt", FILE_WRITE);
    if(dataFile) {
      dataFile.print("X: "); dataFile.print(event.acceleration.x); dataFile.print("  ");
      dataFile.print("Y: "); dataFile.print(event.acceleration.y); dataFile.print("  ");
      dataFile.print("Z: "); dataFile.print(event.acceleration.z); dataFile.print("  ");
    }
    dataFile.close();

  // Accelerometer code end
```

```
load1= (hx711.read()/500.0)-error_load_cell;
Serial.print("Load Cell 1 ");
Serial.println(load1,2);
dataFile = SD.open("load_fr.txt", FILE_WRITE);
if(dataFile) {
  dataFile.print(load1,2);
}
dataFile.close();
load2 = (hx711_2.read()/500.0)-error_load_cell_2;
Serial.print("Load Cell 2 ");
Serial.println(load2,2);
dataFile = SD.open("load_fl.txt", FILE_WRITE);
if(dataFile) {
  dataFile.print(load2,2);
}
dataFile.close();

load3 = (hx711_3.read()/500.0)-error_load_cell_3;
Serial.print("Load Cell 3 ");
Serial.println(load3,2);
dataFile = SD.open("load_tr.txt", FILE_WRITE);
if(dataFile) {
  dataFile.print(load3,2);
}
dataFile.close();

load4 = (hx711_4.read()/500.0)-error_load_cell_4;
Serial.print("Load Cell 4 ");
```

```
  Serial.println(load4,2);
  dataFile = SD.open("load_tl.txt", FILE_WRITE);
  if(dataFile) {
    dataFile.print(load4,2);
  }
  dataFile.close();
  //Load cell code end
  String thermistor = "       ";

  String request = "/exec?load_fr=" + String(load1,2) + String("&load_fl=") + load2 +
String("&load_tr=") + load3 + String("&load_tl=") + load4 + String("&acceleration_x=") +
event.acceleration.x+ String("&acceleration_y=") + event.acceleration.y+
String("&acceleration_z=") + event.acceleration.z + thermistor;
  Serial.println(request);
  espSerial.println(request);

  delay(500);
}
```

Acceleration Visualization code:

```
% read acceleration data from Excel sheet

accel_data = readtable('DAS_data.xlsx');

% extract acceleration data for x, y, z axes
x_acc= accel_data.X;
y_acc= accel_data.Y;
z_acc= accel_data.Z;

% Define constants
mass = 1000; % kg
```

```
radius = 0.25; % m
gear_ratio = 3.5;
efficiency = 0.9;

% Convert acceleration to torque
torque = (mass * (x_acc.^2 + y_acc.^2 + z_acc.^2).^(1/2) * radius) / (gear_ratio * efficiency);

% Plot torque over time
time = (1:length(torque))';
plot(time, torque);
xlabel('Time');
ylabel('Torque (Nm)');
% Load data from Excel
data = readtable('DAS_data.xlsx');
x_acc = data.X;
y_acc = data.Y;
z_acc = data.Z;

% Define constants
mass = 1000; % kg
radius = 0.25; % m
gear_ratio = 3.5;
efficiency = 0.9;

% Convert acceleration to torque
torque = (mass * (x_acc.^2 + y_acc.^2 + z_acc.^2).^(1/2) * radius) / (gear_ratio * efficiency);

% Plot torque over time
time = (1:length(torque))';
```

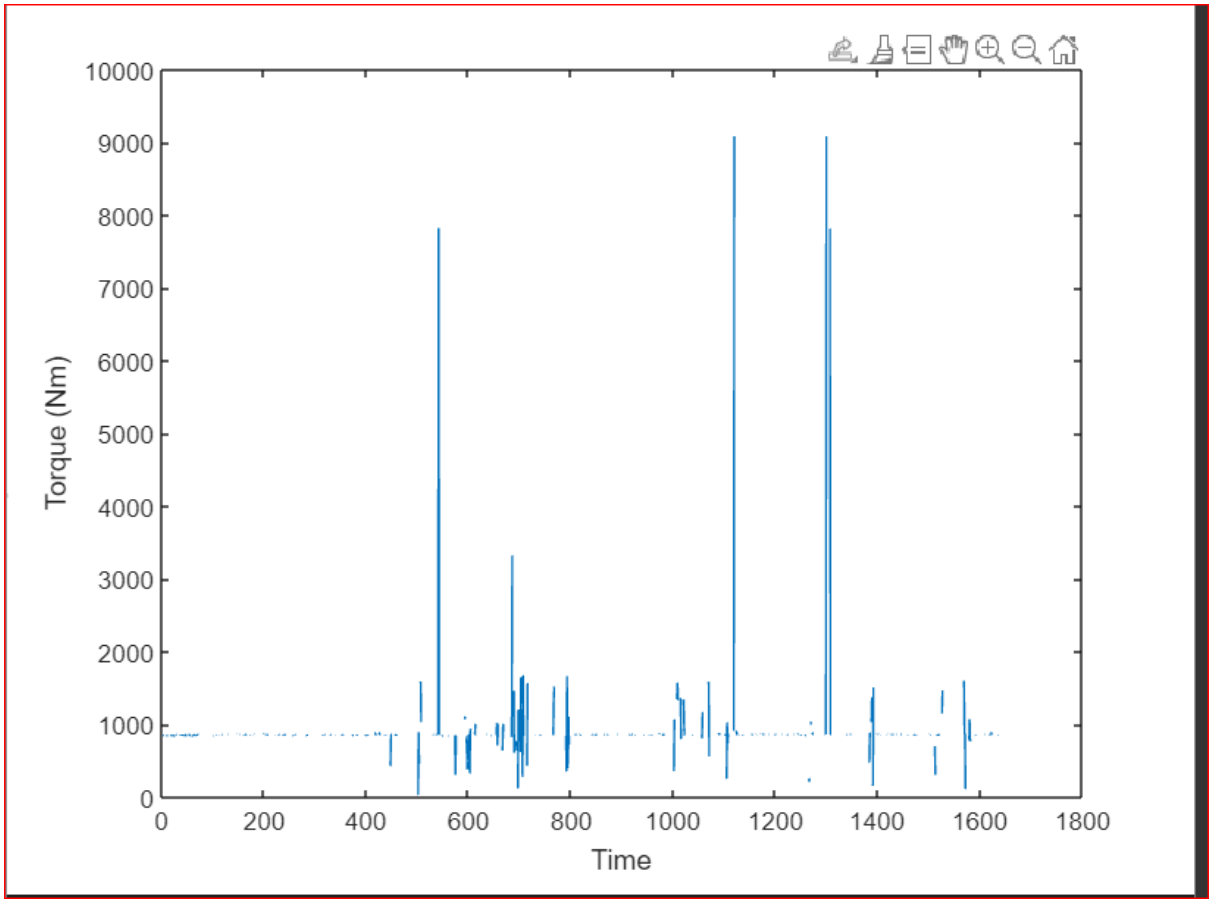
```
plot(time, torque);
xlabel('Time');
ylabel('Torque (Nm)');
```

Load cell data Visualization:

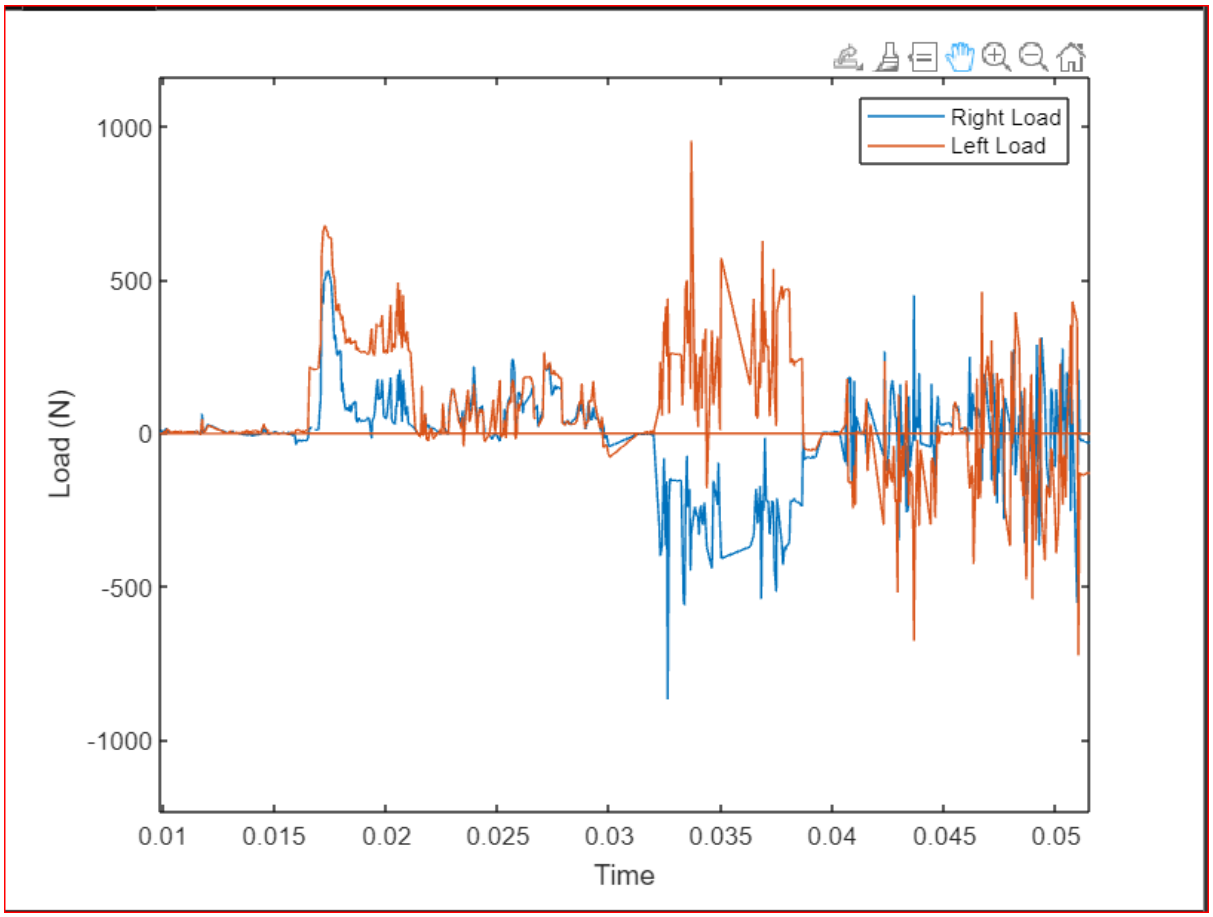
```
% Load data from Excel sheet
data = readtable('DAS_data.xlsx');

% Extract data for right and left load cells
right_load = data.R;
left_load = data.L;


% Plot load cells data over time
time = data.Time;
plot(time, right_load, time, left_load);
xlabel('Time');
ylabel('Load (N)');
legend('Right Load', 'Left Load');
```

Visualizations and Graphs :

Torque Vs time graph



Load Vs time Graph



Implementation :

The implementation of this project involved installing the sensors in the car, connecting them to the Arduino Mega, and configuring the Arduino Mega and ESP8266 modules for data acquisition and Visualization . The data is then analyzed to provide insights into the car's performance. The observations from this project can be used to optimize the car's performance and improve its efficiency .

Inference :

- Validation of Suspension Assembly : while taking Multiple Turns on same side there is more load on one of the wheel . So we need to Validate Maximum Load on the System to avoid breaking of Tierod .

- As we can see the Maximum load observed in 500N in the graph of Load cell vs time but we have designed it with 2KN considering safety parameters .
  Thus Validating the suspension assembly

Challenges faced :

1. We tried to get the live data of temperature node also on the google sheet but while interfacing of the Arduino mega of the temperature sensor node , we had to give an interrupt to the host node and to do this we had to create a bigger delay , first to transmit data from temperature node to host node then the host node reads the value of load cells and accelerometer then it transmits it to the ESP8266 for further transmitting data to the google docs.
2. First we were using thinkspeak cloud platform for data display and data storage but this is not a good option between we were facing a lot of lag in receiving the data and this was not good as our data was time sensitive and for getting better features of thinkspeak we would have to buy the premium version which has enhanced speed and analysis.
3. After shifting to google cloud , which was very fast compared to thinkspeak and this storage was much better. But again for analysis we needed to buy the premium version. We could afford to buy the premium version since it was only 2 rs , but it required us to change a certain setting in the Debit card which was not possible.
4. The issue with GPS module was that serial communication was not possible between ardunio uno and mega , the readings were having a very big time lag and since this was time sensitive data we couldn't afford that.

Conclusion

In conclusion, this project demonstrates the use of Arduino Mega and ESP8266 to acquire data from sensors installed in a car and transmit that data to the Google Cloud Platform for analysis. Making educated decisions about maintenance and optimisation is made possible by the data collected, which offers useful insights into the performance of the vehicle.